

80. Ce que le compilateur refuse, et pourquoi

Le dernier tutoriel n'apprend pas à écrire : il apprend à lire les refus.

1. Ce qu'il faut comprendre

C'est la règle qui gouverne le projet. Un `long` traduit en douce sur un octet, c'est un jeu qui compte faux à partir de 256 et personne n'a été prévenu ; un refus, c'est trois secondes perdues et une ligne à réécrire.

Cinq refus qu'on rencontre vraiment. `float vitesse = 1.5;` — la machine n'a pas de virgule. `class Ennemi { ... };` — le message propose `struct`, qui fait ce qu'on voulait, méthodes comprises. `carte[3] = 1;` sur une grille — il faut **deux** index, car `carte[3]` désigne une rangée entière. `uint8_t loin(Ennemi e)` — un argument est un octet ; une `struct` se passe par son nom, visible partout, ou par son index. `TABLE[0] = 9;` sur une table `const` — elle est gravée dans la cartouche.

Chacun porte **son numéro de ligne**, et dit quoi écrire à la place. C'est ce qui fait la différence entre un compilateur qui aide et un compilateur qui punit : les deux disent non, un seul dit pourquoi. Le programme ci-dessous est la version acceptée des cinq.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1442 octets** de cartouche.

1. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi { uint8_t x, y; };
```

2. UNE TABLE GRAVÉE DANS LA CARTOUCHE — TABLE

```
const uint8_t TABLE[] = { 1, 2, 3 };
```

3. UN TABLEAU — CARTE

```
uint8_t carte[4][5];
```

4. UNE VARIABLE — BOSS

```
Ennemi boss;  
  
/* Pas la struct : son INDEX. Un argument est un octet. */
```

5. UNE FONCTION — LOIN()

```
uint8_t loin(uint8_t cible) {  
    if (boss.x > cible) return boss.x - cible;  
    return cible - boss.x;  
}
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    boss.x = 40;
    carte[2][3] = 1;           // deux index, pas un

    uint8_t vitesse = 2;       // un octet, pas un float

    texte(1, 4, "ECART");
    nombre(9, 4, loin(10));

    texte(1, 6, "GRILLE");
    nombre(9, 6, carte[2][3], 1);

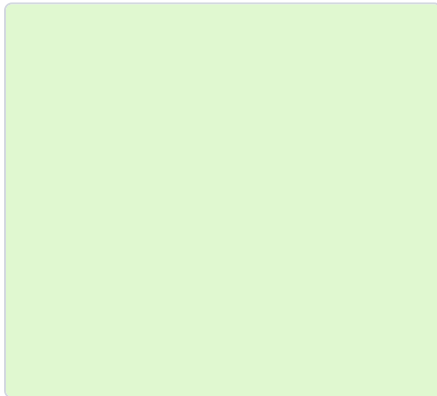
    texte(1, 8, "TABLE");
    nombre(9, 8, TABLE[2], 1);

    texte(1, 10, "PAS");
    nombre(9, 10, vitesse, 1);

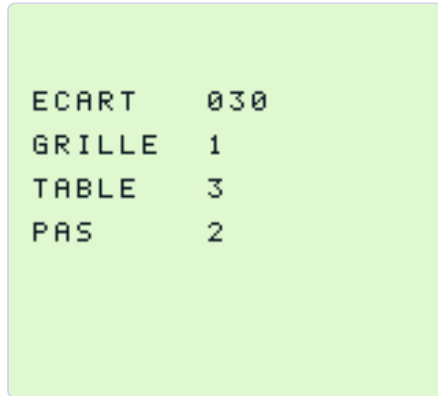
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

030, 1, 3, 2 — les cinq refus, écrits comme il faut.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la struct passe par son index

✓ la grille prend ses deux index

✓ la table gravée se lit

✓ et l'octet remplace le float

Ensuite : **81. Ce qu'une ligne de C++ devient, en vrai** — niveau 10 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++